

# FlowAutoTTS: Automatic Test-Time Scaling for Flow Matching Samplers

Jikun Cai\*, & Yutong Huang\*, Kehan Lan\*  
Fudan University

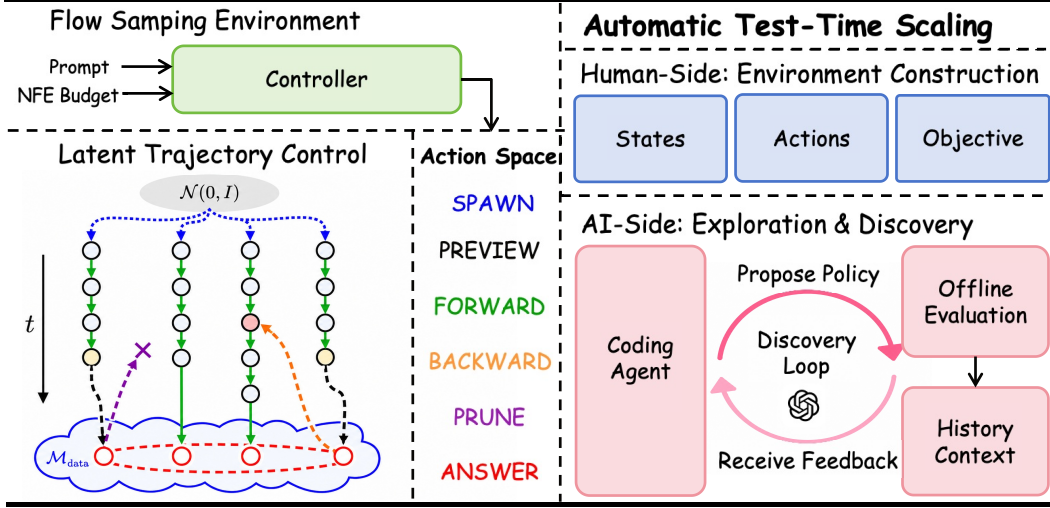


Figure 1: Overview of **FlowAutoTTS**. We formulate flow-matching inference as a controller-driven process over latent trajectories. A controller interacts with the environment through a small set of actions. An explorer agent then iteratively proposes executable controller programs, evaluates them under different NFE budgets, receives feedback as context, and improves the controller to optimize the reward–NFE frontier.

## Abstract

Flow-matching samplers generate data by transporting latent variables from noise to data through a learned velocity field and are widely used in today’s Text-to-Image models. Recent inference-time strategies improve such samplers with hand-crafted test-time scaling (TTS) mechanisms (e.g. best-of-N selection). Inspired by recent progress in LLM-based algorithm discovery and automatic heuristic design, we propose **FlowAutoTTS**, an agentic framework for automatically discovering test-time scaling controllers for flow-matching samplers. We formulate flow-matching inference as a controller-driven process over latent trajectories. A controller interacts with a compact sampling environment through a small set of actions, while an explorer agent iteratively proposes controller programs, receives feedback as context, and improves the controller to optimize the reward–NFE Pareto frontier. We instantiate FlowAutoTTS on Stable Diffusion 3.5 Medium and evaluate it under multiple inference budgets. Across preference alignment and compositional generation benchmarks, including PickScore, OCR, and GenEval, the discovered controllers achieve stronger reward–NFE frontiers than simple baselines in 5 rounds of evolution. These results suggest that test-time scaling for flow-matching models can be automated by constructing environments for agents to discover and raise broader insights about the capability boundaries of today’s agentic systems in algorithm design. Our implementation code is at <https://github.com/Eric-dot-pixel/FlowEvo>.

\*Equal Contribution.

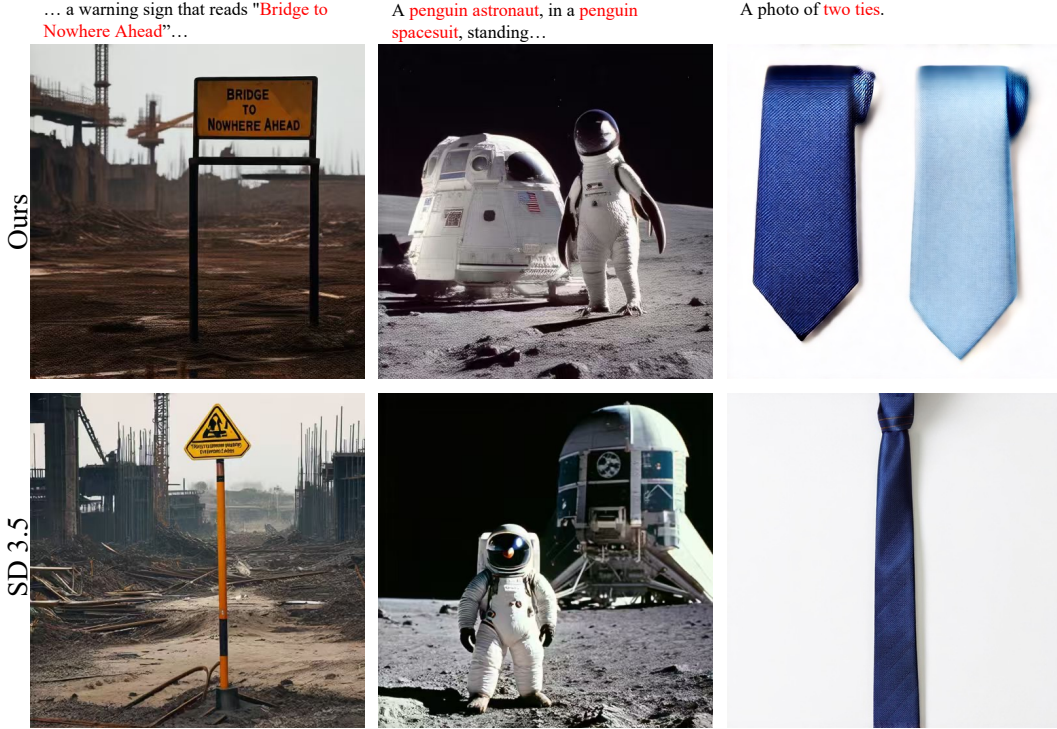


Figure 2: Qualitative comparison with SD 3.5 Med (using naive ODE sampler). FlowAutoTTS better preserves text rendering, rare composition, and object-count requirements.

## 1 Introduction

Flow matching (Lipman et al., 2022; Liu et al., 2022) has become an important formulation for modern generative modeling. It learns a time-dependent velocity field that transports samples from a simple noise distribution to the data distribution. At inference time, a sampler numerically integrates this learned vector field, producing a latent trajectory from noise to image. This formulation has been adopted by recent large-scale text-to-image systems such as Stable Diffusion 3 (Esser et al., 2024), which combines the flow-matching objective with diffusion transformer backbones (Peebles & Xie, 2023). In common implementations, a standard baseline is the deterministic Euler sampler, where each step advances the current latent according to the predicted velocity field.

A natural question is how to improve such samplers efficiently without retraining the base model. Recent inference-time strategies, including best-of- $N$  selection, noise re-injection and stochastic branching show that additional test-time computation can improve generation quality and alignment (Jang et al., 2026; Ma et al., 2025; Zhou et al., 2026; Xu et al., 2023; Bai et al., 2025). However, these methods are still largely hand-crafted. Their key decisions, such as when to sample more candidates, inject noise, and prune weak trajectories, are manually specified through algorithm-specific rules and hyperparameters. This makes each new strategy dependent on engineering efforts and leaves a broader space unexplored.

Our key observation is that many existing inference-time strategies for flow-matching samplers can be viewed as special cases of a common latent-trajectory control problem. Although these procedures differ in motivation and implementation, they can be approximately expressed using a small set of primitive actions over latent states like moving forward in trajectory or re-noise backward stochastically. This perspective suggests that, instead of manually designing a single heuristic sampler, we can define an environment with a set of actions and search for effective controllers within it.

Meanwhile, recent agentic systems such as FunSearch (Romera-Paredes et al., 2024), Evolution of Heuristics (Liu et al., 2024), AlphaEvolve (Novikov et al., 2025) and ADAS (Hu et al., 2025) use language models to propose executable programs, evaluate them with task-specific feedback, and iteratively improve the discovered algorithms. More directly related to our work, Zheng et al. (2026) formulate test-time scaling for language models as environment-driven controller discovery, where an explorer agent searches over branching, probing, pruning, and stopping policies using execution history as feedback. This reframes test-time scaling from heuristic design into environment design and make us wonder if it could be transferred to the TTS strategies of flow-matching sampling.

Motivated by this view, we propose **FlowAutoTTS**, an agentic framework for automatically discovering test-time scaling controllers for flow-matching samplers. As illustrated in Fig. 1, we formulate flow-matching inference as a controller-driven process over latent trajectories. A controller interacts with the environment through a small set of actions, including SPAWN, FORWARD, PREVIEW, BACKWARD, PRUNE, and ANSWER. These actions allow the controller to allocate inference compute across trajectories, inspect intermediate clean predictions, refine or diversify latent states, discard unpromising candidates, and choose the final output. An explorer agent then iteratively proposes executable controller programs, evaluates them under different NFE budgets, receives feedback as context, and improves the controller to optimize the reward–NFE Pareto frontier.

We instantiate FlowAutoTTS on Stable Diffusion 3.5 Medium and evaluate the discovered controllers under multiple inference budgets. Across preference alignment and compositional generation benchmarks, including PickScore (Kirstain et al., 2023), OCR-oriented generation (Cui et al., 2025), and GenEval (Ghosh et al., 2023), FlowAutoTTS discovers controllers that achieve stronger reward–NFE frontiers than simple baselines such as the deterministic FlowMatch Euler sampler and best-of- $N$  selection after only five rounds of evolution, as is shown in Fig 2. These results suggest that test-time scaling for flow-matching models can be automated by constructing environments for agents to discover within. More broadly, FlowAutoTTS provides a concrete setting for probing the capability boundaries of contemporary agentic systems in algorithm design.

Our contributions are summarized as follows:

- We formulate inference-time scaling for flow-matching samplers as a latent-trajectory control problem, where existing hand-crafted strategies can be estimated as special cases of primitive actions over latent states.
- We propose **FlowAutoTTS**, an environment-driven framework in which an explorer agent automatically discovers controller programs for allocating sampling compute under different NFE budgets.
- We show on Stable Diffusion 3.5 Medium that the discovered controllers improve the reward–NFE Pareto frontier over deterministic FlowMatch Euler sampling and best-of- $N$  selection across PickScore, OCR-oriented generation, and GenEval.

## 2 Preliminaries: Flow Matching

We use  $\mathbf{z}_t \in \mathbb{R}^d$  to denote the latent variable at time  $t \in [0, 1]$ , where  $t = 0$  corresponds to noise and  $t = 1$  corresponds to data. Let  $p_0$  be a simple prior distribution, typically  $\mathcal{N}(\mathbf{0}, \mathbf{I})$ , and let  $p_1$  be the data distribution. Given a condition  $c$ , e.g., a text prompt, flow matching learns a time-dependent velocity field that transports  $p_0$  to  $p_1$ .

Let  $\mathbf{z}_0 \sim p_0$  and  $\mathbf{z}_1 \sim p_1$ . We define noised data using the Rectified Flow (Liu et al., 2022)

$$\mathbf{z}_t = (1 - t)\mathbf{z}_0 + t\mathbf{z}_1, \quad t \in [0, 1]. \quad (1)$$

The corresponding target velocity is

$$\mathbf{u}_t(\mathbf{z}_t \mid \mathbf{z}_0, \mathbf{z}_1) = \frac{d\mathbf{z}_t}{dt} = \mathbf{z}_1 - \mathbf{z}_0. \quad (2)$$

A conditional flow model  $\mathbf{u}_\theta(\mathbf{z}_t, t, c)$ , parameterized by  $\theta$ , is trained to predict this velocity:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, \mathbf{z}_0, \mathbf{z}_1, c} \left[ \left\| \mathbf{u}_\theta(\mathbf{z}_t, t, c) - (\mathbf{z}_1 - \mathbf{z}_0) \right\|_2^2 \right]. \quad (3)$$

At inference time, generation is performed by solving the probability-flow ODE

$$\frac{d\mathbf{z}_t}{dt} = \mathbf{u}_\theta(\mathbf{z}_t, t, \mathbf{c}), \quad \mathbf{z}_0 \sim p_0, \quad (4)$$

from  $t = 0$  to  $t = 1$ . With a time grid  $0 = t_0 < t_1 < \dots < t_K = 1$  and step size  $\Delta t_i = t_{i+1} - t_i$ , the Euler update is

$$\mathbf{z}_{t_{i+1}} = \mathbf{z}_{t_i} + \Delta t_i \mathbf{u}_\theta(\mathbf{z}_{t_i}, t_i, \mathbf{c}). \quad (5)$$

This deterministic ODE sampler is efficient, but for a fixed initial noise  $\mathbf{z}_0$  it produces only one trajectory, which limits stochastic exploration during test-time scaling.

**Stochastic sampling.** To introduce randomness while preserving the same marginal path  $\{p_t\}_{t \in [0,1]}$ , the ODE can be replaced by a marginal-preserving SDE following [Liu et al. \(2026\)](#):

$$d\mathbf{z}_t = \left[ \mathbf{u}_\theta(\mathbf{z}_t, t, \mathbf{c}) + \frac{1}{2} \sigma_t^2 \nabla_{\mathbf{z}} \log p_t(\mathbf{z}_t) \right] dt + \sigma_t d\mathbf{w}_t. \quad (6)$$

Here  $\mathbf{w}_t$  is a standard Wiener process,  $\sigma_t \geq 0$  is a time-dependent diffusion coefficient, and  $\nabla_{\mathbf{z}} \log p_t(\mathbf{z}_t)$  is the score of the intermediate marginal distribution. When  $\sigma_t = 0$ , the SDE reduces to the deterministic ODE. When  $\sigma_t > 0$ , the sampler injects stochasticity while keeping the same marginal distribution  $p_t$  at each time.

A first-order discretization gives the Euler–Maruyama update

$$\mathbf{z}_{t_{i+1}} = \mathbf{z}_{t_i} + \Delta t_i \left[ \mathbf{u}_\theta(\mathbf{z}_{t_i}, t_i, \mathbf{c}) + \frac{1}{2} \sigma_{t_i}^2 \nabla_{\mathbf{z}} \log p_{t_i}(\mathbf{z}_{t_i}) \right] + \sigma_{t_i} \sqrt{\Delta t_i} \boldsymbol{\epsilon}_i, \quad \boldsymbol{\epsilon}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (7)$$

Compared with deterministic ODE sampling, stochastic sampling can generate multiple particles from the same intermediate state. This property is useful for inference-time exploration, where additional computation can be allocated to branch, perturb, refine, or select among candidate trajectories.

### 3 Flow-Matching Test-Time Scaling as Latent-Trajectory Control

We formulate test-time scaling for flow-matching samplers as a controller problem over latent trajectories. Given a prompt condition  $\mathbf{c}$  and an NFE budget  $B$ , the controller interacts with an online sampling environment. At each controller step  $k$ , the environment state is

$$\mathbf{s}_k = (\mathbf{c}, B_k, \mathcal{I}_k, \mathcal{C}_k, \mathcal{P}_k, \{P_i\}_i, \{A_j\}_j, H_k), \quad (8)$$

where  $B_k$  is the remaining NFE budget,  $\mathcal{I}_k, \mathcal{C}_k$ , and  $\mathcal{P}_k$  denote the active, completed, and pruned particle sets, respectively,  $P_i$  is a particle record,  $A_j$  is a preview anchor, and  $H_k$  is the event history observed by the controller.

Each particle record stores

$$P_i = (\mathbf{z}_i, \tau_i, \text{parent}_i, \text{anchor}_i, \text{nfe}_i, \text{status}_i), \quad (9)$$

where  $\mathbf{z}_i$  is the current latent,  $\tau_i \in [0,1]$  is its current flow time, and  $\text{status}_i \in \{\text{active}, \text{completed}, \text{pruned}\}$ . Each preview anchor stores

$$A_j = (i, \tau_i, \hat{\mathbf{z}}_{1,j}, \hat{\mathbf{z}}_{0,j}, r_j), \quad (10)$$

where  $\hat{\mathbf{z}}_{1,j}$  is a clean-endpoint estimate,  $\hat{\mathbf{z}}_{0,j}$  is an inferred noise anchor,  $r_j$  is a reward score.

Finally, we define the admissible action set

$$\mathcal{A}(\mathbf{s}) = \mathcal{A}_{\text{SPAWN}} \cup \mathcal{A}_{\text{FORWARD}} \cup \mathcal{A}_{\text{PREVIEW}} \cup \mathcal{A}_{\text{BACKWARD}} \cup \mathcal{A}_{\text{PRUNE}} \cup \mathcal{A}_{\text{ANSWER}}. \quad (11)$$

**SPAWN.** The action

$$\text{SPAWN}(n) \quad (12)$$

creates  $n$  new active particles from the prior distribution:

$$\mathbf{z}_i \sim p_0, \quad \tau_i = 0. \quad (13)$$

This action increases the search width.

**FORWARD.** The action

$$\text{FORWARD}(i, \tau', \text{solver}), \quad \tau' > \tau_i, \quad (14)$$

advances an active particle  $i$  from time  $\tau_i$  to a later time  $\tau'$ . The solver can use ODE or SDE.

**PREVIEW.** The action

$$\text{PREVIEW}(i) \quad (15)$$

queries the model at the current particle state and constructs both clean and noise anchors:

$$\hat{\mathbf{z}}_{1,i} = \mathbf{z}_i + (1 - \tau_i) \mathbf{u}_\theta(\mathbf{z}_i, \tau_i, \mathbf{c}), \quad \hat{\mathbf{z}}_{0,i} = \mathbf{z}_i - \tau_i \mathbf{u}_\theta(\mathbf{z}_i, \tau_i, \mathbf{c}). \quad (16)$$

The clean anchor may be decoded and evaluated by a reward model:

$$r_i = R(D(\hat{\mathbf{z}}_{1,i}), \mathbf{c}), \quad (17)$$

where  $D$  is the decoder and  $R$  is a verifier or reward function. The preview action does not move the particle itself. Instead, it reveals an anchor and its corresponding score as an estimator of the quality of the current latent. It is crucial to the explorer because it provides *intermediate signals* for the current trajectory and guides the evolution of the controllers.

**BACKWARD.** The action

$$\text{BACKWARD}(j, \tau', \pi_{\text{noise}}, m, M, \eta) \quad (18)$$

creates  $m$  child particles from preview anchor  $A_j$ <sup>1</sup> by re-noising its clean endpoint estimate to target time  $\tau' \in [0, 1]$ . For each child, the environment first samples a noise anchor

$$\tilde{\mathbf{z}}_0^{(\ell)} \sim \pi_{\text{noise}}(\cdot | \hat{\mathbf{z}}_{0,j}, \eta), \quad \ell = 1, \dots, m. \quad (19)$$

The implementation supports three noise policies:

$$\begin{aligned} \text{FRESH\_NOISE} : \quad \tilde{\mathbf{z}}_0^{(\ell)} &\sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \\ \text{INFERRED\_NOISE} : \quad \tilde{\mathbf{z}}_0^{(\ell)} &= \hat{\mathbf{z}}_{0,j}, \\ \text{MIXED\_NOISE} : \quad \tilde{\mathbf{z}}_0^{(\ell)} &= \eta \hat{\mathbf{z}}_{0,j} + (1 - \eta) \boldsymbol{\epsilon}^{(\ell)}, \quad \boldsymbol{\epsilon}^{(\ell)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \end{aligned} \quad (20)$$

Given the sampled noise anchor, the child latent is constructed by the same linear interpolation used by the flow path:

$$\mathbf{z}_{\tau'}^{(\ell)} = (1 - \tau') \tilde{\mathbf{z}}_0^{(\ell)} + \tau' \hat{\mathbf{z}}_{1,j}. \quad (21)$$

**PRUNE.** The action

$$\text{PRUNE}(\mathcal{S}), \quad \mathcal{S} \subseteq \mathcal{I}_k, \quad (22)$$

marks a subset of active particles as pruned:

$$\mathcal{I}_{k+1} = \mathcal{I}_k \setminus \mathcal{S}. \quad (23)$$

Pruned particles are no longer eligible for future actions.

**ANSWER.** The action

$$\text{ANSWER}(\rho) \quad (24)$$

terminates the episode and returns a final latent according to an answer rule  $\rho$ . In our implementation,  $\rho$  may choose the clean anchor with the best preview score, or finish the latest active particle and score its final sample:

$$\hat{\mathbf{x}} = D(\hat{\mathbf{z}}_\rho). \quad (25)$$

Existing TTS methods approximately correspond to manually specified programs in our action space (especially using BACKWARD). FlowAutoTTS keeps the environment fixed and delegates the search for such programs to an explorer agent.

<sup>1</sup>This design is a compromise. One could define a stochastic reverse action that directly maps a particle from  $\mathbf{z}_t$  to a target time  $t' < t$ . Our BACKWARD is forced to operate on a preview anchor i.e. after PREVIEW. This anchor-based re-noising is generally not equivalent to a direct stochastic reverse transition from  $\mathbf{z}_t$  to the same target time  $t'$ . We adopt this design because the preview anchor provides an observable signal for deciding whether, where, and how strongly to branch backward. If not used, the BACKWARD action would become an unguided stochastic perturbation.

## 4 Automatic Controller Evolution

**Optimization Objective.** Let  $\beta \in [\beta_{\min}, \beta_{\max}]$  denote the test-time NFE budget. A controller is a budget-conditioned program

$$\pi(\cdot \mid s, \beta), \quad (26)$$

which interacts with the sampling environment and returns a final image for a prompt  $c \sim \mathcal{D}$ :

$$\hat{\mathbf{x}}_\pi(c, \beta). \quad (27)$$

The output is evaluated by a reward function

$$R(\hat{\mathbf{x}}_\pi(c, \beta), c). \quad (28)$$

For each budget  $\beta$ , the expected reward of controller  $\pi$  is

$$F_\pi(\beta) = \mathbb{E}_{c \sim \mathcal{D}} [R(\hat{\mathbf{x}}_\pi(c, \beta), c)]. \quad (29)$$

Thus, a single controller induces a reward–budget curve  $\beta \mapsto F_\pi(\beta)$ .

The ideal reward–NFE frontier is the pointwise upper envelope over all controllers:

$$F^*(\beta) = \sup_{\pi \in \Pi} F_\pi(\beta). \quad (30)$$

Our goal is to discover a single budget-conditioned controller whose induced curve approaches this frontier. We formalize this by minimizing the weighted frontier regret

$$\text{Regret}(\pi) = \int_{\beta_{\min}}^{\beta_{\max}} w(\beta) [F^*(\beta) - F_\pi(\beta)] d\beta, \quad (31)$$

where  $w(\beta) > 0$  specifies the importance of different budget regimes. Since  $F^*(\beta)$  is independent of  $\pi$ , minimizing this regret is equivalent to maximizing the weighted area under the controller’s reward–budget curve:

$$\mathcal{J}(\pi) = \int_{\beta_{\min}}^{\beta_{\max}} w(\beta) F_\pi(\beta) d\beta. \quad (32)$$

This objective does not assume that a single controller can attain  $F^*(\beta)$  at every budget; rather, it provides a practical surrogate for approaching the frontier across budgets.

In practice, we evaluate a fixed discrete budget grid  $\{\beta_m\}_{m=1}^M$  and a finite search prompt set  $\{c_n\}_{n=1}^N$ . The objective is approximated as

$$\hat{\mathcal{J}}(\pi) = \sum_{m=1}^M w_m \frac{1}{N} \sum_{n=1}^N R(\hat{\mathbf{x}}_\pi(c_n, \beta_m), c_n). \quad (33)$$

**Automatic Evolution.** Let  $\pi^{(r)}$  denote the controller proposed at evolution round  $r$ . The explorer agent maintains a history of evaluated controllers, including their reward–NFE curves, scalarized scores, and execution traces. At each round, the explorer proposes a new executable controller  $\pi^{(r)}$ , evaluates it on the fixed budget grid and training set prompts, receives  $\hat{\mathcal{J}}(\pi^{(r)})$  together with execution-level feedback, and uses this history as context to propose the next controller  $\pi^{(r+1)}$ . We prompt the explorer to achieve a better frontier.

## 5 Experiments

**Implementation Details.** We use Stable Diffusion 3.5 Medium as the base flow-matching text-to-image model and GPT-5.4-thinking as the explorer agent. For each benchmark, the controller is evolved for five rounds on 500 training samples. The controller takes a scalar budget parameter  $\beta$  as input, which is mapped to a discrete NFE budget:

$$\beta \in \{0, 0.25, 0.5, 0.75, 1.0\} \mapsto \text{NFE} \in \{10, 20, 36, 48, 64\}. \quad (34)$$

All controllers are evaluated under the same budget grid.

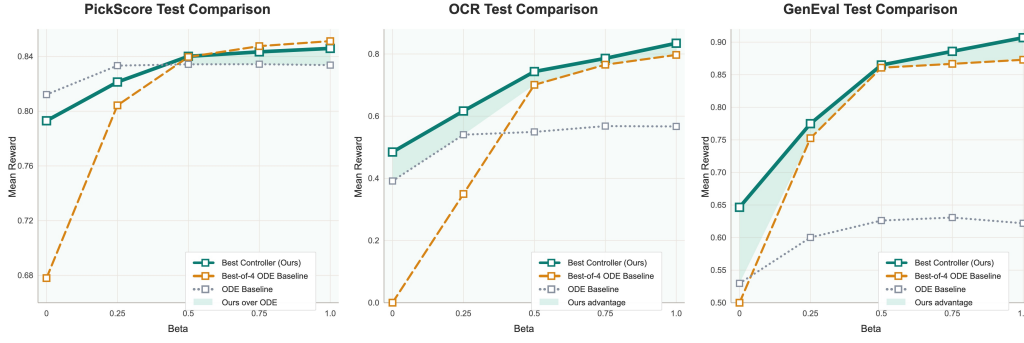


Figure 3: Overall quantitative results across PickScore, OCR, and GenEval. Curves compare FlowAutoTTS with ODE and best-of-4 baselines under increasing NFE budgets.

Table 1: Training-set rewards across benchmarks and budget levels. Each cell reports the reward under the corresponding benchmark and  $\beta$  budget.

| Method    | PickScore |       |       |       |       | OCR       |       |       |       |       | GenEval   |       |       |       |       |
|-----------|-----------|-------|-------|-------|-------|-----------|-------|-------|-------|-------|-----------|-------|-------|-------|-------|
|           | $\beta=0$ | .25   | .5    | .75   | 1.0   | $\beta=0$ | .25   | .5    | .75   | 1.0   | $\beta=0$ | .25   | .5    | .75   | 1.0   |
| Euler     | 80.94     | 82.83 | 82.94 | 82.92 | 82.94 | 39.67     | 55.13 | 55.76 | 57.85 | 57.38 | 48.82     | 52.87 | 54.93 | 56.52 | 56.78 |
| Best-of-4 | 67.65     | 80.21 | 83.66 | 84.40 | 84.73 | 00.02     | 35.68 | 70.69 | 76.10 | 81.83 | 01.20     | 67.32 | 80.47 | 79.65 | 81.18 |
| Round 0   | 77.81     | 78.97 | 83.90 | 84.16 | 84.46 | 35.81     | 59.20 | 70.18 | 77.65 | 80.71 | 29.87     | 61.87 | 76.88 | 80.32 | 82.48 |
| Round 1   | 77.81     | 78.97 | 83.80 | 84.06 | 82.92 | 35.81     | 52.15 | 67.04 | 74.06 | 79.28 | 57.80     | 64.58 | 76.95 | 80.78 | 83.68 |
| Round 2   | 78.80     | 80.14 | 83.69 | 83.41 | 84.38 | 47.67     | 55.34 | 62.63 | 65.41 | 68.82 | 49.93     | 64.58 | 75.83 | 81.23 | 83.73 |
| Round 3   | 78.80     | 81.79 | 83.21 | 84.06 | 84.34 | 48.07     | 55.27 | 70.24 | 76.83 | 80.86 | 55.72     | 64.58 | 78.63 | 81.08 | 84.30 |
| Round 4   | 78.80     | 81.79 | 83.74 | 84.01 | 84.30 | 47.67     | 61.60 | 75.17 | 78.75 | 83.15 | 55.72     | 68.42 | 78.67 | 81.28 | 84.40 |

**Benchmarks & Baselines.** We evaluate FlowAutoTTS on three benchmarks. **PickScore** measures human-preference alignment between prompts and generated images. **OCR** evaluates the model’s ability to render readable and correct text inside images. **GenEval** measures compositional text-to-image alignment, including object presence, counting, colors, co-occurrence, and spatial relations. We compare against two simple baselines: a deterministic FlowMatch Euler sampler, which follows a single ODE trajectory, and best-of- $N$  selection with  $N = 4$  (floor dividing), which generates multiple independent candidates and selects the one with the highest benchmark score. These baselines represent depth-only sampling and simple width scaling, respectively.

**Evolution On Training-set.** Table 1 summarizes the training-set rewards across benchmarks and budget levels. The evolved controllers bring clear improvements over the initial controller, particularly on OCR and GenEval. On OCR, Round 4 improves over Round 0 at all budget levels, increasing the reward from 35.81 to 47.67 at  $\beta=0$  and from 70.18 to 75.17 at  $\beta=0.5$ . It also outperforms Euler and Best-of-4 on most OCR settings. Similarly, on GenEval, Round 4 achieves strong gains over Round 0, especially under low budgets, and reaches the best reward of 84.40 at  $\beta=1.0$ . PickScore is more saturated, since most methods already obtain high rewards when  $\beta \geq 0.5$ ; nevertheless, the evolved controllers remain competitive with Best-of-4 and consistently outperform Euler at higher budget levels. These results suggest that evolution is most effective on benchmarks with larger optimization room, while preserving strong performance on preference-based rewards.

**Results on Test-set.** We also tested the performance of the best controllers (Round 4) on each benchmark’s test set. Figure 3 visualizes the scaling curves of our Round 5 best controllers v.s. baselines on each benchmark. FlowAutoTTS gives larger gains on OCR and GenEval, while PickScore is nearly saturated across budget. It turns out that the automatic algorithm searches in our pre-defined action space make better generalizable controllers

which make better use of different NFE budgets and generally achieve better frontiers. We also note that since previous studies on generative model test-time-scaling (Ma et al., 2025), different searching algorithms fit different tasks. So we did not perform cross-benchmark evaluation of different controllers due to limited computing resources.

## 6 Conclusion

In this work, we formulate Flow Matching samplers’ Test Time Scaling as an controller design problem on latent trajectories and proposed FlowAutoTTS, an agentic framework that design better controllers based on history context to achieve better reward-NFE frontiers on Text-to-Image generation tasks.

**Limitations.** Due to limited resources, we did not compare many strong baselines e.g. P&P (Jang et al., 2026). Moreover, our maximum NFE budget is only set to 64, which cannot be a very solid evaluation setting for test time scaling and the final controllers are benchmark-conditioned and hard to explain.

**Discussion.** However, we show that the strong capabilities today’s LLMs have demonstrated in heuristic algorithm design, which is to some extent evaluated on the Flow Matching sampler design task by our work after we managed to provide an environment and define a problem. Our motivation also comes from a recent insightful blog on Heuristic Learning (Weng, 2026). We conclude that with the LLMs evolving stronger and larger, our focus might be shifted from designing specific programs or algorithms to designing environments for LLM agents to understand, explore and discover.

## References

- Lichen Bai, Shitong Shao, Zipeng Qi, Zhiqiang Xu, Haoyi Xiong, Zeke Xie, et al. Zigzag diffusion sampling: Diffusion models can self-improve via self-reflection. In *International Conference on Learning Representations*, volume 2025, pp. 38561–38591, 2025.
- Cheng Cui, Ting Sun, Manhui Lin, Tingquan Gao, Yubo Zhang, Jiaxuan Liu, Xueqing Wang, Zelun Zhang, Changda Zhou, Hongen Liu, et al. Paddleocr 3.0 technical report. *arXiv preprint arXiv:2507.05595*, 2025.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first international conference on machine learning*, 2024.
- Dhruba Ghosh, Hannaneh Hajishirzi, and Ludwig Schmidt. Geneval: An object-focused framework for evaluating text-to-image alignment. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 52132–52152. Curran Associates, Inc., 2023. URL [https://proceedings.neurips.cc/paper\\_files/paper/2023/file/a3bf71c7c63f0c3bcb7ff67c67b1e7b1-Paper-Datasets\\_and\\_Benchmarks.pdf](https://proceedings.neurips.cc/paper_files/paper/2023/file/a3bf71c7c63f0c3bcb7ff67c67b1e7b1-Paper-Datasets_and_Benchmarks.pdf).
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, volume 2025, pp. 21344–21377, 2025.
- Sangwon Jang, Taekyung Ki, Jaehyeong Jo, Saining Xie, Jaehong Yoon, and Sung Ju Hwang. Self-refining video sampling. *arXiv preprint arXiv:2601.18577*, 2026.
- Yuval Kirstain, Adam Polyak, Uriel Singer, Shahbuland Matiana, Joe Penna, and Omer Levy. Pick-a-pic: An open dataset of user preferences for text-to-image generation. *Advances in neural information processing systems*, 36:36652–36663, 2023.
- Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

- Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. *arXiv preprint arXiv:2401.02051*, 2024.
- Jie Liu, Gongye Liu, Jiajun Liang, Yangguang Li, Jiaheng Liu, Xintao Wang, Pengfei Wan, Di Zhang, and Wanli Ouyang. Flow-grpo: Training flow matching models via online rl. *Advances in neural information processing systems*, 38:40783–40818, 2026.
- Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.
- Nanye Ma, Shangyuan Tong, Haolin Jia, Hexiang Hu, Yu-Chuan Su, Mingda Zhang, Xuan Yang, Yandong Li, Tommi Jaakkola, Xuhui Jia, et al. Scaling inference time compute for diffusion models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 2523–2534, 2025.
- Alexander Novikov, Ng  n V  , Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 4195–4205, 2023.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- Jiayi Weng. Learning beyond gradients. <https://trinkle23897.github.io/learning-beyond-gradients/>, May 2026. Blog post.
- Yilun Xu, Mingyang Deng, Xiang Cheng, Yonglong Tian, Ziming Liu, and Tommi Jaakkola. Restart sampling for improving generative processes. *Advances in Neural Information Processing Systems*, 36:76806–76838, 2023.
- Tong Zheng, Haolin Liu, Chengsong Huang, Huiwen Bao, Sheng Zhang, Rui Liu, Runpeng Dai, Ruibo Chen, Chenxi Liu, Tianyi Xiong, et al. Llms improving llms: Agentic discovery for test-time scaling. *arXiv preprint arXiv:2605.08083*, 2026.
- Zikai Zhou, Muyao Wang, Shitong Shao, Lichen Bai, Haoyi Xiong, Bo Han, and Zeke Xie. Reflective flow sampling enhancement. *arXiv preprint arXiv:2603.06165*, 2026.